ANVAYA NOVA

AI-Based Agricultural Produce Quality Grading, Price Estimation and Value-Addition Recommendation

COMPLETE ENGINEERING PROJECT REPORT

Submitted by: [STUDENT NAME]

USN / Registration No.: [USN]

Guide: [GUIDE NAME, DESIGNATION]

Department: [DEPARTMENT NAME]

College / Institution: [COLLEGE NAME]

Academic year: [ACADEMIC YEAR]

Implementation and latest evaluation evidence reviewed through 23 October 2026

CERTIFICATE

This is to certify that the engineering project report entitled “AI-Based Agricultural Produce Quality Grading, Price Estimation and Value-Addition Recommendation” is a record of work carried out by [STUDENT NAME], USN [USN], under my supervision, in partial fulfilment of [PROGRAM / DEGREE] at [COLLEGE NAME] during [ACADEMIC YEAR]. The implementation status and results are based on the project files and runs reviewed for this report.

Guide: [GUIDE NAME] Signature: _____ Date: _____
Head of Department: [HOD NAME] Signature: _____

DECLARATION

I, [STUDENT NAME], USN [USN], declare that this report is prepared from the project implementation and cited sources. Implemented functionality is distinguished from proposed functionality, and the reported results are not represented as a new training run.

Place: [PLACE] Date: [DATE] Student signature: _____

ACKNOWLEDGEMENT

I thank [GUIDE NAME], [DEPARTMENT NAME], [COLLEGE NAME], faculty, colleagues, and family for support. I acknowledge the open-source PyTorch, torchvision, scikit-learn, OpenCV, Pillow, NumPy, and Matplotlib communities. Dataset attribution and permissions must be completed by the author; these were not documented in the inspected repository.

ABSTRACT

This report documents Anvaya Nova, a tomato-focused computer-vision prototype for fresh-versus-rotten classification and A/B/C visual quality grading. Both models use torchvision EfficientNet-B0 with pretrained weights; the feature extractor is frozen and a task-specific linear classifier is trained with Adam and cross-entropy. The repository includes image-quality screening and single- and three-image grading workflows. Evaluation on existing checkpoints produced 95.33% accuracy and 95.38% weighted F1 on 300 binary test images, and 82.28% accuracy and 82.35% weighted F1 on 79 grading test images. Price estimation, farmer-to-business and consumer connections, and value-addition recommendations are proposals, not implemented features. Results do not establish field reliability or food safety.

TABLE OF CONTENTS

Certificate
Declaration
Acknowledgement
Abstract
List of Figures and Tables
Chapter 1: Introduction
Chapter 2: Literature Survey
Chapter 3: Problem Statement, Objectives, and Scope
Chapter 4: Existing System and Proposed System
Chapter 5: System Requirements and Architecture
Chapter 6: Dataset Collection and Preprocessing
Chapter 7: Methodology and Algorithms
Chapter 8: System Implementation
Chapter 9: Results and Performance Evaluation
Chapter 10: Market Price Estimation and Value-Addition Recommendations
Chapter 11: Limitations and Future Enhancements
Chapter 12: Conclusion
References

LIST OF FIGURES AND TABLES

Figures: 1. System architecture and implementation boundary; 2. Training and evaluation data flow; 3. Single-image inference flowchart.
Tables: 1. Dataset split counts and mappings; 2. Environment; 3. Fresh/rotten results; 4. A/B/C results; 5. Feature implementation status.

Table 1. Dataset split counts and mappings

Task / split	Class counts	Total
Binary train	fresh 944; rotten 445	1,389
Binary validation	fresh 202; rotten 95	297
Binary test	fresh 204; rotten 96	300
Grading train	A 101; B 161; C 88	350
Grading validation	A 21; B 34; C 18	73
Grading test	A 23; B 36; C 20	79

Table 2. Environment and observed runtime

Component	Observed
Python	3.13.14
PyTorch / torchvision	2.14.0 / 0.29.0
scikit-learn / OpenCV	1.9.1 / 5.0.0.93
Evaluation device	CPU
Resize / batch	224x224 / 16

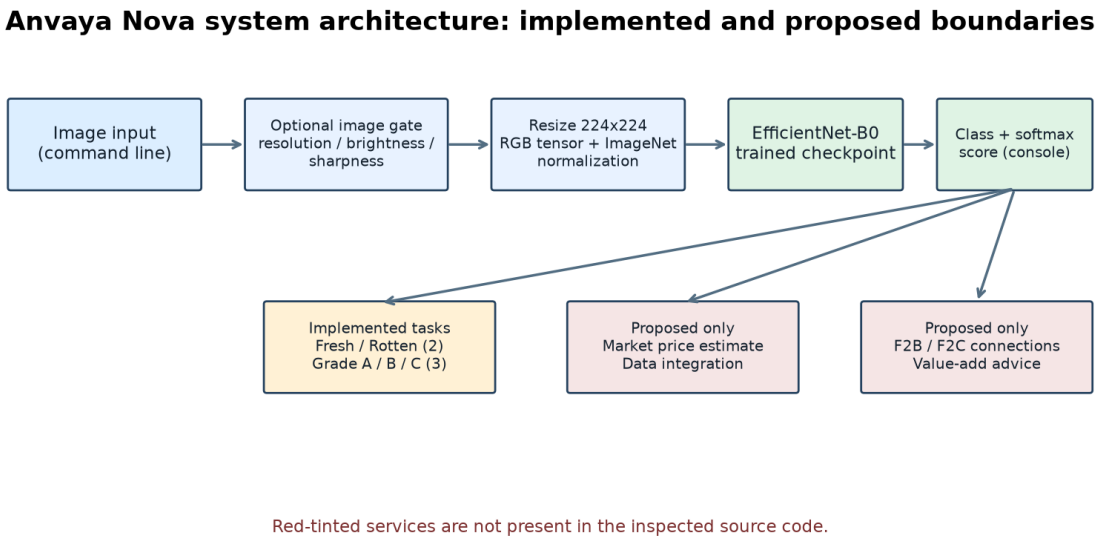


Figure 1. System architecture; proposed services are not implemented.

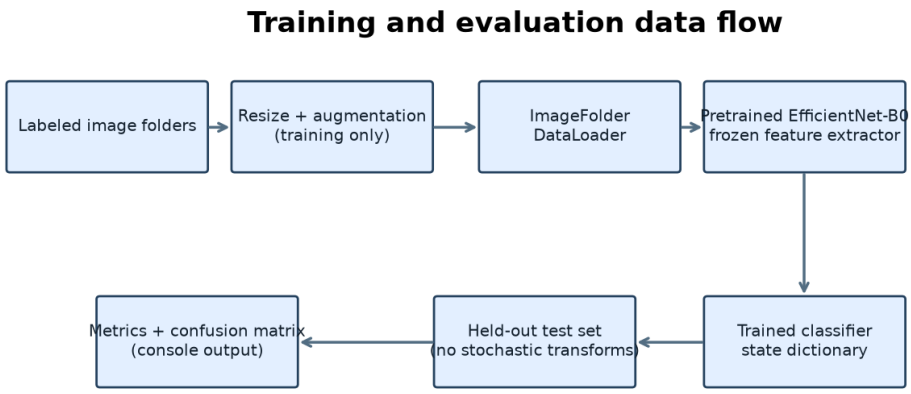


Figure 2. Training and evaluation data flow.

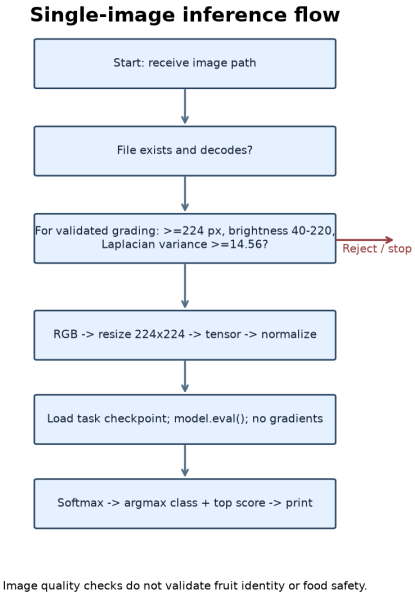


Figure 3. Single-image inference flowchart.

Chapter 1: Introduction

1.1 Background

Agricultural produce is often assessed by visual inspection. Manual sorting can vary between evaluators and may be slow when handling large lots. Computer vision can assist with repeatable image-based classification, while a wider digital service could connect quality information with price discovery and post-harvest decisions.

1.2 Project context

Anvaya Nova is a prototype for tomato image analysis. The inspected implementation contains two trained image-classification tasks: fresh-versus-rotten prediction and Grade A/B/C visual quality grading. A separate image-validation function screens for file readability, minimum resolution, brightness, and sharpness before the grading workflow. The codebase does not currently implement market-price estimation, marketplace connections, or value-addition recommendation; these are proposed extensions described separately.

1.3 Motivation

A low-cost image workflow can support preliminary sorting and demonstrate how visual AI might assist agricultural decisions. It must be treated as decision support, not a substitute for food-safety inspection, regulatory grading, or expert assessment.

1.4 Report basis

This report is grounded in the source code, directory counts, grading rules, saved model files, and the latest successful local evaluation runs inspected on 23 October 2026. No dataset origin beyond information visible in the project files is assumed. No training logs or independently recorded run metadata were found.

Chapter 2: Literature Survey

2.1 Convolutional neural networks

Convolutional neural networks learn hierarchical spatial features from images. Residual networks showed that shortcut connections can support training of deeper networks [2]. For this project, an ImageNet-pretrained EfficientNet-B0 is used as the visual backbone.

2.2 EfficientNet

EfficientNet scales network depth, width, and input resolution using a compound scaling strategy. EfficientNet-B0 is the baseline architecture in the family [1]. The project uses torchvision's pretrained EfficientNet-B0 and replaces its final linear layer for the project-specific class count.

2.3 Transfer learning

Transfer learning reuses representations learned on a broad source dataset and adapts a task-specific output head. The project freezes pretrained parameters and optimizes the replacement classifier head with Adam and cross-entropy. This is feature-extractor transfer learning; the inspected scripts do not fine-tune the feature backbone.

2.4 Augmentation and evaluation

Image augmentation can improve robustness by exposing a model to label-preserving visual variation [3]. Training applies horizontal flips and small rotations; grading additionally uses brightness, contrast, and saturation jitter. Accuracy alone can conceal class-specific errors, so precision, recall, F1-score, and a confusion matrix are also reported [4].

2.5 Agricultural image classification

Prior agricultural imaging research has explored deep learning for plant-disease recognition, including controlled-dataset studies [5]. Such results do not establish the performance of this tomato dataset or deployment domain. The current project evaluates only its own held-out image folders.

2.6 Gap addressed

The prototype demonstrates two tomato classification tasks and a deterministic image-quality gate. It does not yet combine grading with verified market data, transaction workflows, or evidence-based processing recommendations. Those elements require additional data, integrations, field validation, and safeguards.

Chapter 3: Problem Statement, Objectives, and Scope

3.1 Problem statement

Manual visual sorting can be inconsistent, while small producers may lack accessible digital tools that connect quality assessment to potential selling or processing pathways. This project addresses the image-classification part of that problem and specifies a future system that may connect classification with price information and value-addition choices.

3.2 Objectives

1) Classify tomato images as fresh or rotten. 2) Assign a visible-quality grade A, B, or C. 3) reject images that fail minimum technical quality checks in the validated workflow. 4) measure held-out classification performance using standard metrics. 5) define an extensible architecture for future price estimation, farmer-to-business/consumer connections, and value-addition advice.

3.3 Implemented scope

Implemented: PyTorch/torchvision EfficientNet-B0 classifiers; ImageFolder-based datasets; model training scripts; evaluation scripts; single-image binary and grading prediction scripts; three-image grade prediction with majority vote; image validation and validated grading prediction; sharpness analysis utility; trained state-dictionary files.

3.4 Excluded or proposed scope

Not implemented in the inspected project: object detection/YOLO, internal spoilage or chemical safety detection, wax-coating detection, live or historical market-price ingestion, price prediction, seller/buyer matching, payment, logistics, consumer listings, or automated value-addition recommendations. These are design proposals only.

3.5 Intended users

The proposed system targets farmers, producer groups, aggregators, processors, retailers, and consumers. At present, the delivered interface is command-line scripts rather than a farmer-facing application.

Chapter 4: Existing System and Proposed System

4.1 Existing practice

Visual grading by people can be practical but varies with experience, lighting, fatigue, and grading criteria. Manual price discovery and buyer search may require separate channels. This report does not quantify those baseline costs because no field study was supplied.

4.2 Current prototype

A user supplies a tomato image to a command-line program. Depending on the entry point, the program loads a saved EfficientNet-B0 state dictionary, resizes and normalizes the image, computes class scores, and prints a class and softmax confidence. The validated path runs technical image checks before invoking the A/B/C classifier.

4.3 Proposed end-to-end service

A future service may accept a batch or lot, record provenance and location, show model grades with uncertainty, query an authorized price feed, present indicative—not guaranteed—price ranges, and offer buyer/consumer channels. Grade C or surplus produce could prompt optional processing pathways such as puree, sauce, drying, or composting only after food-safety, shelf-life, equipment, and local regulatory checks. No recommendation engine exists today.

4.4 Feature status

Implemented now: tomato visual classification, technical image validation in one workflow, console output, three-view grade aggregation. Proposed: mobile/web user interface, lot/traceability records, live prices, price estimation, marketplace listings, industry matching, consumer connections, logistics, value-addition recommendations, analytics, and deployment monitoring.

Table 5. Implemented versus proposed feature status

Feature	Status
Fresh/rotten and A/B/C classification	Implemented
Image-quality gate and validated grading	Implemented
Three-image grade voting	Implemented; unvalidated
Market price / live feed	Proposed only
Farmer-to-business / consumer	Proposed only
Value-addition recommendations	Proposed only
YOLO, internal spoilage, wax detection	Not implemented

Chapter 5: System Requirements and Architecture

5.1 Observed software stack

Python 3.13.14 selected environment; PyTorch 2.14.0; torchvision 0.29.0; scikit-learn 1.9.1; OpenCV 5.0.0.93; Pillow 12.3.0; NumPy 2.5.3; Matplotlib 3.11.2. The environment details were read from the project venv. Actual hardware used for the latest evaluation was CPU, as printed by both evaluation scripts.

5.2 Functional requirements

FR-1 load an image path; FR-2 preprocess RGB images to 224x224 and normalize using ImageNet channel statistics; FR-3 predict fresh/rotten or A/B/C using the corresponding saved model; FR-4 evaluate the held-out sets and calculate classification metrics; FR-5 optionally gate grading on image resolution, brightness, and sharpness. Proposed FR-6 to FR-9 cover price data, farmer/business/consumer connections, and recommendations.

5.3 Non-functional requirements

Reproducibility requires documented data provenance, versioned splits, model/configuration tracking, and deterministic evaluation. Safety requires warnings that visible appearance cannot establish internal food safety. Privacy requires consent and access controls if location, identity, or transaction data are later collected. Availability, latency, accessibility, and security have not been formally measured.

5.4 Architecture

The current architecture is a local Python command-line pipeline: image file -> optional OpenCV quality gate -> Pillow/torchvision transform -> EfficientNet-B0 classifier -> class/confidence output. Evaluation uses ImageFolder test data -> same normalization -> saved checkpoint -> metrics/confusion matrix. Proposed modules are visually distinguished in Figure 1.

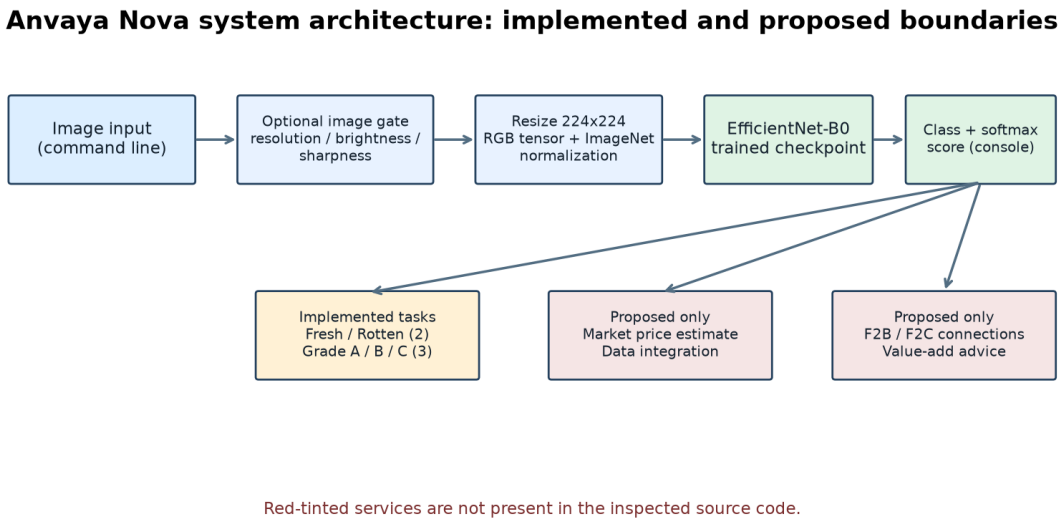


Figure 1. Implemented local pipeline and proposed services.

5.5 Hardware

The code automatically selects CUDA when available and otherwise CPU. Latest test output selected CPU. No minimum RAM, GPU, camera, or production-server requirement was specified or benchmarked.

Chapter 6: Dataset Collection and Preprocessing

6.1 Dataset evidence and counts

The project contains image folders for two separate tasks. The fresh/rotten split contains 1,389 train images (944 fresh, 445 rotten), 297 validation images (202 fresh, 95 rotten), and 300 test images (204 fresh, 96 rotten), 1,986 split images total. The grading split contains 350 train (101 A, 161 B, 88 C), 73 validation (21 A, 34 B, 18 C), and 79 test (23 A, 36 B, 20 C), 502 images total. A separate raw grading folder also contains 502 images; it is a source copy and must not be added to the split count. Dataset provenance, collection protocol, consent, geographic representation, and image-level duplication checks are not documented.

6.2 Class mappings

ImageFolder assigns class indices alphabetically by subdirectory: binary classes ['fresh', 'rotten']; grading classes ['A', 'B', 'C']. The binary prediction script uses ['fresh', 'rotten']; grade prediction scripts use ['A', 'B', 'C']. The grading rules file describes visible appearance: A is uniform with no significant defects, B allows minor defects, and C represents significant visible defects. Those text rules are guidance, not formal regulatory standards.

6.3 Split generation

The split scripts use a 70%/15%/15% train/validation/test ratio, random.seed(42), shuffle file lists, and copy images. The grading splitter applies integer truncation independently per grade; observed counts are A 101/21/23, B 161/34/36, C 88/18/20. The binary script uses a user Downloads path as source and the same ratios. Existing output folders are not cleared before copying, so reruns against changed sources can leave stale files; this is a reproducibility risk.

Task	Split	Counts	Total
Binary	Train	944 fresh / 445 rotten	1,389
Binary	Validation	202 fresh / 95 rotten	297
Binary	Test	204 fresh / 96 rotten	300
A/B/C	Train	101 A / 161 B / 88 C	350
A/B/C	Validation	21 A / 34 B / 18 C	73
A/B/C	Test	23 A / 36 B / 20 C	79

6.4 Transformations

Both training pipelines resize to 224x224, convert to tensors, and normalize RGB channels with mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. Binary training adds RandomHorizontalFlip and RandomRotation(10). Grading training adds RandomHorizontalFlip, RandomRotation(15), and ColorJitter with brightness/contrast/saturation 0.2. Validation, test, and inference use resize, tensor conversion, and normalization without stochastic augmentation.

6.5 Data quality caveats

The image quality checker uses OpenCV BGR input and grayscale conversion. It requires width and height >=224, mean grayscale brightness from 40 through 220, and Laplacian variance >=14.56. Thresholds are hard-coded and there is no captured per-image validation output in the repository. It cannot establish that a tomato is present or that the fruit is internally sound.

Chapter 7: Methodology and Algorithms

7.1 EfficientNet-B0 and transfer learning

EfficientNet-B0 is instantiated from torchvision's pretrained DEFAULT weights in both training scripts. The binary task replaces classifier[1] with a two-output linear layer and freezes all model parameters before replacement; the grading task freezes model.features and replaces classifier[1] with a three-output layer. Adam (learning rate 0.001) updates only the replacement classifier layer. CrossEntropyLoss trains the model. Binary training runs five epochs; grading training runs eight. Batch size is 16. Training outputs are saved as state dictionaries.

7.2 Binary classification algorithm

Input image -> RGB conversion -> 224x224 resize -> tensor -> ImageNet normalization -> EfficientNet-B0 -> two logits -> softmax probabilities -> argmax -> fresh/rotten label and top-class confidence. The model class output relies on ImageFolder alphabetical order. Softmax confidence is not calibrated probability of correctness.

7.3 Grading algorithm

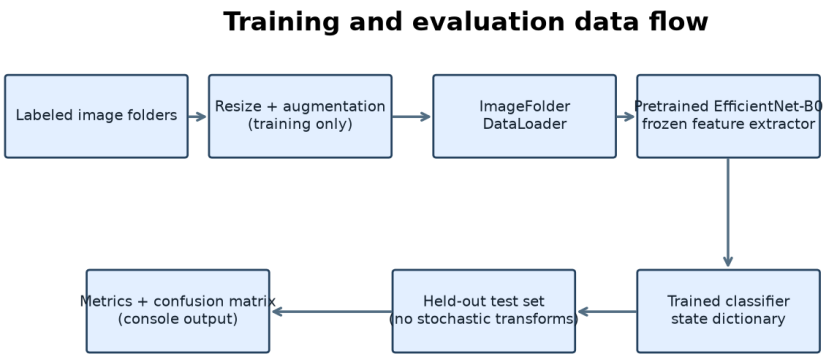
The same inference preprocessing feeds a three-output classifier. The maximum softmax class maps by output index to A, B, or C; scripts print the top grade and per-grade probabilities. The model predicts a visual dataset label; it does not apply explicit defect measurements or a written rule engine.

7.4 Three-image grading

multi_predict_grading.py requires exactly three image paths, predicts each separately, counts A/B/C predictions, selects the most frequent grade, and reports the mean probability for that selected grade as combined confidence. Python max selects the first encountered key on ties (A before B before C given insertion order). This is a prototype aggregation rule, not a statistically validated multi-view method.

7.5 Evaluation

The evaluation scripts use non-shuffled batches of 16, torch.no_grad(), and sklearn accuracy, weighted precision, weighted recall, weighted F1, and confusion_matrix. The grading evaluation additionally prints classification_report. Weighted metrics average class metrics by true support. Confusion-matrix rows are true classes and columns are predicted classes in the alphabetical class order.



Fresh/rotten and A/B/C tasks use separate datasets and checkpoints.

Figure 2. Training and evaluation data flow.

7.6 Pseudocode

TRAIN: load labeled train/validation folders; build transforms/loaders; instantiate ImageNet-pretrained EfficientNet-B0; freeze feature parameters; replace final linear layer with K outputs; optimize head with Adam and cross-entropy for configured epochs while monitoring accuracy; save state_dict. INFER: open RGB image; apply deterministic transform; run model in eval/no-grad mode; softmax logits; return argmax class and score. VALIDATE: check file; decode; check dimensions; compute grayscale mean and Laplacian variance; reject on threshold failure; otherwise allow downstream grading.

Chapter 8: System Implementation

8.1 Source inventory

Present source files: train.py, evaluate.py, predict.py, train_grading.py, evaluate_grading.py, predict_grading.py, multi_predict_grading.py, image_validation.py, predict_validated.py, check_sharpness.py, split_dataset.py, and split_grading_dataset.py. No project documentation or run logs were found in the inspected non-venv source tree.

8.2 Training and checkpoint files

tomato_quality_model.pth and tomato_grading_model.pth are present in the repository root. They contain state dictionaries according to the save/load calls in the scripts. They were loaded successfully by the latest evaluation runs. Training was not re-run for this report; the available source defines training configuration, while the measured results are from evaluation of the existing checkpoint files.

8.3 Inference entry points

predict.py performs fresh/rotten prediction. predict_grading.py performs single-image A/B/C grading. multi_predict_grading.py requires exactly three images. predict_validated.py calls validate_image and then grades an accepted image. Evaluation and training scripts assume they are launched from the project root because paths are relative.

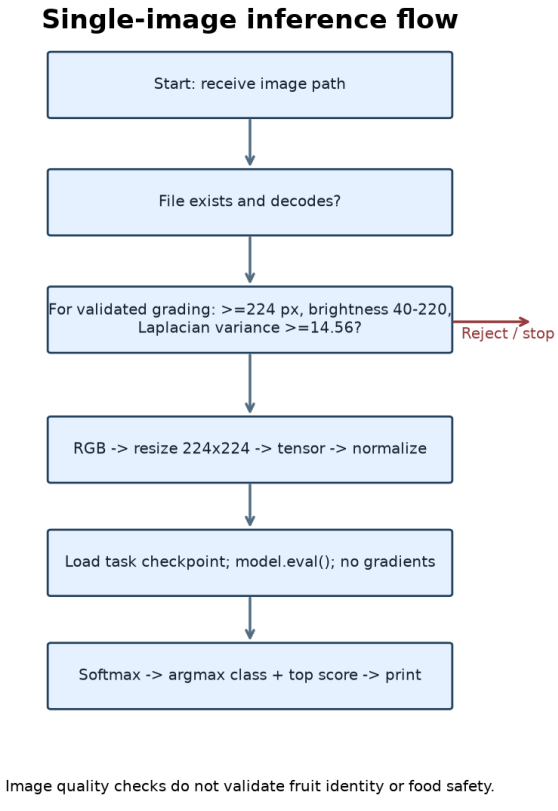


Figure 3. Inference and optional validation.

8.4 Input validation and operational behavior

predict.py and grading scripts print errors for unreadable images, but argument handling generally calls sys.exit() without a nonzero code. image_validation.py returns False for expected file/decode/quality failures and the validated predictor stops before model inference. No web service, database, authentication, UI, or

price service is present.

8.5 Reproduction commands

From C:\\Users\\Dell\\Agriculture AI: .venv\\Scripts\\python.exe src\\evaluate.py; .venv\\Scripts\\python.exe src\\evaluate_grading.py; .venv\\Scripts\\python.exe src\\predict_validated.py \"path-to-image.jpg\". The two evaluation commands were actually executed for this report; prediction examples are commands inferred from the CLI interface and were not run against new images.

Chapter 9: Results and Performance Evaluation

9.1 Evaluation conditions

Both evaluation scripts were executed using the project's Python 3.13.14 virtual environment on 23 October 2026. Their console output selected CPU. Binary testing used dataset/tomato/test (300 images); grading testing used dataset/tomato/grading_split/test (79 images). These are observed current-run results, not results from retraining or an independent external benchmark.

9.2 Fresh-versus-rotten results

Accuracy = 95.33%; weighted precision = 95.57%; weighted recall = 95.33%; weighted F1 = 95.38%. Confusion matrix [[193, 11], [3, 93]], rows/columns ordered fresh, rotten. Thus fresh: 193 correct, 11 predicted rotten; rotten: 93 correct, 3 predicted fresh. Test supports: 204 fresh and 96 rotten.

Metric	Result
Accuracy	95.33%
Weighted precision	95.57%
Weighted recall	95.33%
Weighted F1	95.38%
Confusion matrix	[[193, 11], [3, 93]]; fresh, rotten

9.3 A/B/C grading results

Accuracy = 82.28%; weighted precision = 83.27%; weighted recall = 82.28%; weighted F1 = 82.35%. Confusion matrix [[20, 3, 0], [7, 27, 2], [1, 1, 18]], rows/columns ordered A, B, C. Per-class report: A precision 0.71, recall 0.87, F1 0.78 (support 23); B precision 0.87, recall 0.75, F1 0.81 (support 36); C precision 0.90, recall 0.90, F1 0.90 (support 20).

Metric	Result
Accuracy	82.28%
Weighted precision	83.27%
Weighted recall	82.28%
Weighted F1	82.35%
Confusion matrix	[[20,3,0],[7,27,2],[1,1,18]]; A, B, C
Per-class F1	A 0.78; B 0.81; C 0.90

9.4 Metric definitions

Accuracy = (TP+TN)/N for binary or correct predictions/N generally. Precision_c = TP_c/(TP_c+FP_c); Recall_c = TP_c/(TP_c+FN_c); F1_c = 2*Precision_c*Recall_c/(Precision_c+Recall_c). Weighted metric = sum_c(support_c * metric_c)/N. Confusion matrix cell M[i,j] counts examples whose true class is i and prediction is j. Values use the observed test outputs; no metrics were estimated or fabricated.

9.5 Interpretation and limitations

The binary model performed better on this test split than the three-grade task. In grading, A and B are most often confused (7 true B images predicted A; 3 true A images predicted B); C achieved 0.90 recall and F1. The test sets are modest and class imbalanced. No confidence intervals, repeated seeds, cross-dataset tests, calibration, environmental robustness, or statistical significance tests were performed. Results should

not be generalized to field deployment.

Chapter 10: Market Price Estimation and Value-Addition Recommendations

10.1 Current status

No price-estimation code, market-price dataset, API client, buyer/seller database, transaction facility, or value-addition recommendation logic was found. This entire chapter describes a proposal, not a feature in the trained models or current application.

10.2 Proposed price estimation

A future price service could ingest verified mandi/market transactions with market, commodity/variety, grade, unit, date/time, and source. It should normalize units and currencies, remove invalid observations with traceable rules, handle seasonality and regional variation, and return a range with timestamp, sample count, source attribution, and uncertainty. One possible baseline is a regularized regression or tree-based model using location, date/season, grade, variety, volume, and market covariates; validation must be time-ordered and market-held-out to avoid leakage. Grade predictions should not be treated as a price without validated calibration and local buyer acceptance.

10.3 Farmer-to-business and farmer-to-consumer

Proposed farmer-to-business (F2B) capabilities: opt-in lot listings, quantity and quality summaries, location at an appropriately consented precision, buyer specifications, indicative offer comparison, and contact/fulfilment workflow. Proposed farmer-to-consumer (F2C) capabilities: consumer-facing listings, traceability, order quantities, pickup/delivery options, and transparent pricing. No matching, messaging, payments, shipping, or buyer verification is implemented. Future deployment needs fraud controls, dispute processes, privacy protections, and clear responsibility boundaries.

10.4 Value-addition recommendation framework

A future rule-based assistant could consider grade distribution, ripeness, volume, time-to-sale, processing capacity, food safety, energy/water, packaging, shelf life, demand, and local regulations. Candidate pathways such as sauce/puree, drying, or composting should be presented only as conditional options requiring human confirmation and a validated processing plan. Grade C does not itself mean safe for processing; visible appearance cannot detect pathogens, toxins, or internal damage.

10.5 Safeguards and evaluation

Proposed price outputs must be marked estimates, not guarantees; identify data recency and provenance; avoid disadvantaging low-volume producers; provide non-digital alternatives; and permit human override. Evaluate price error (MAE/RMSE), interval coverage, freshness, regional fairness, buyer conversion, realized net farmer revenue, waste avoided, and recommendation safety through a prospective pilot.

Chapter 11: Limitations and Future Enhancements

11.1 Current limitations

Tomato-only labels; undocumented data provenance and consent; relatively small evaluation sets; no external validation; visual labels do not establish safety; fixed image-quality thresholds; no calibration or uncertainty handling; no formal grade-standard mapping; hard-coded relative paths; no robust nonzero CLI exit conventions; grading split reruns can retain stale files; the three-image majority-vote method is unvalidated; no market module or user interface.

11.2 Model and data improvements

Record dataset source, capture device, conditions, location and permissions; deduplicate by source/fruit and split by collection session to reduce leakage; establish expert-reviewed labels and inter-rater agreement; collect more balanced, diverse field data; compare frozen-feature extraction with controlled fine-tuning; tune thresholds only on validation data; report per-class metrics, confidence intervals, calibration, and external-site tests.

11.3 Product improvements

Build a mobile-friendly interface with offline capture guidance, batch/lot workflow, multilingual accessibility, audit trails, and clear limitations. Add secure, permissioned data storage and monitoring. Introduce market integration only after selecting authorized data sources and validating freshness. Add F2B/F2C features and value-addition decision support as separately tested modules.

11.4 Safety boundary

The current classifier evaluates image appearance only. It does not detect internal spoilage, chemical residues, microbial hazards, wax, or food safety. Product safety must follow applicable inspection and processing controls; AI grades must not be presented as a safety certification.

Chapter 12: Conclusion

12.1 Summary

Anvaya Nova currently provides a functional local prototype for tomato fresh/rotten classification and A/B/C visual grading using transfer-learned EfficientNet-B0 models. It also provides image quality validation and a three-image grading aggregation utility. The latest measured held-out results are 95.33% binary accuracy and 82.28% grading accuracy on their respective project test folders.

12.2 Final assessment

The implementation establishes a credible technical baseline, but performance evidence is limited to small project-specific test sets. It does not yet estimate prices, connect farmers to businesses or consumers, or recommend value addition. Those requirements remain future engineering work requiring market data, user research, safety governance, and field trials.

12.3 Closing

The project can be advanced responsibly by improving dataset traceability and external validation first, then delivering a human-centered interface, followed by separately validated market and recommendation modules. All model outputs should remain advisory until their performance and real-world consequences are evaluated.

REFERENCES

[1] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. 36th Int. Conf. Machine Learning (ICML), PMLR, vol. 97, 2019, pp. 6105–6114.

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[3] C. Shorten and T. M. Khoshgoftaar, “A survey on Image Data Augmentation for Deep Learning,” Journal of Big Data, vol. 6, art. 60, 2019, doi: 10.1186/s40537-019-0197-0.

[4] D. M. W. Powers, “Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation,” Journal of Machine Learning Technologies, vol. 2, no. 1, pp. 37–63, 2011.

[5] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection,” Frontiers in Plant Science, vol. 7, art. 1419, 2016, doi: 10.3389/fpls.2016.01419.

[6] PyTorch, “torchvision.models.efficientnet_b0,” torchvision documentation. Available: https://pytorch.org/vision/stable/models/generated/torchvision.models.efficientnet_b0.html. Accessed: Oct. 23, 2026.

[7] scikit-learn developers, “Model evaluation: quantifying the quality of predictions,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed: Oct. 23, 2026.

MANUAL COMPLETION CHECKLIST

Replace all bracketed personal and institutional placeholders; verify degree, dates, signature and certificate requirements; add dataset provenance, license and permissions; and confirm reference style required by the institution.